

Vehicle Rental Price Calculation System

Assignment 2 — Microservices Architecture

CCCS-425-764 Web Services

Summer 2026

McGill University, School of Continuing Studies

Brad Cibula

Student ID: 260295088

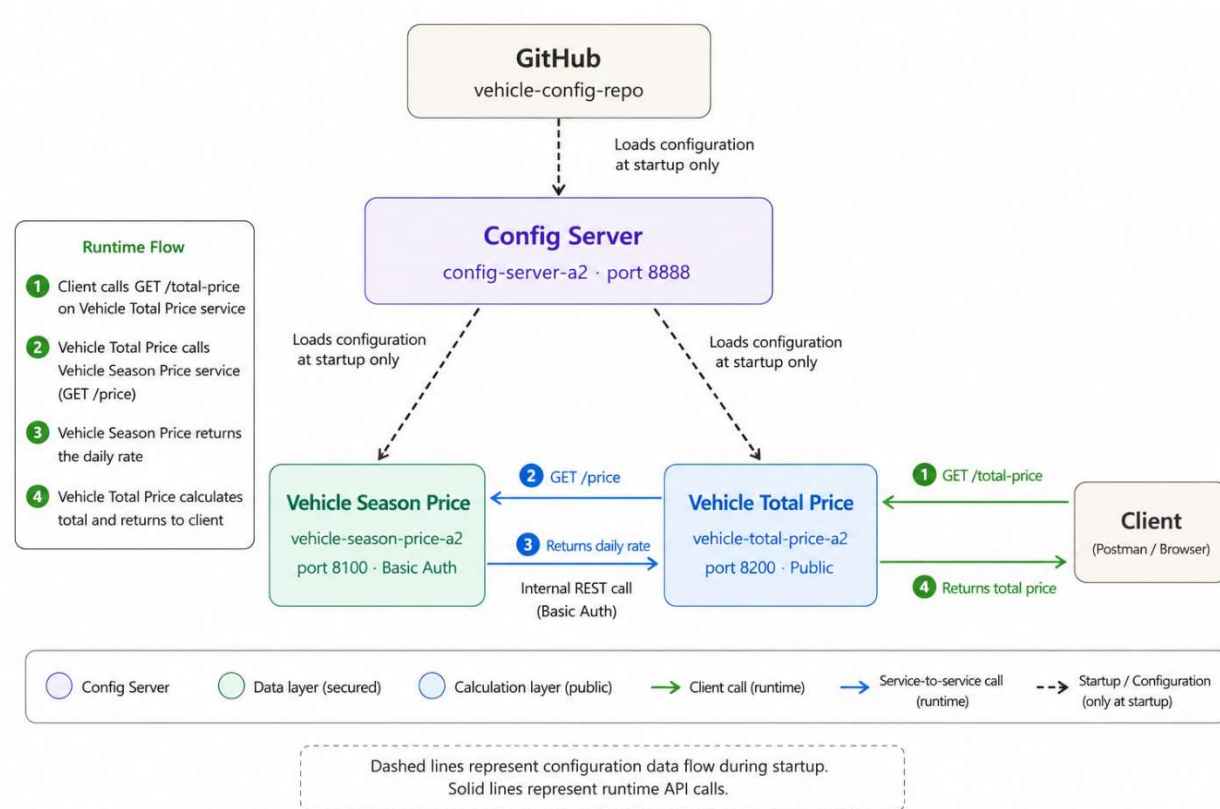
August, 2026

1. Architecture Overview

This project implements a Vehicle Rental Price Calculation System using a microservices architecture built on Spring Boot 4.x and Spring Cloud. The system consists of three components: a centralized configuration server and two independently deployable microservices that communicate over HTTP.

1.1 Components

Component	Artifact	Port	Role
Config Server	config-server-a2	8888	Serves configuration to all services from GitHub
Vehicle Season Price	vehicle-season-price-a2	8100	Returns daily rental rate for a vehicle and season
Vehicle Total Price	vehicle-total-price-a2	8200	Calls season-price service and calculates total cost



1.2 Service Interaction

The Vehicle **Total** Price microservice acts as the *calculation* layer. It receives a request containing a vehicle type, season, and number of rental days. It then calls the Vehicle Season Price microservice to retrieve the daily rate, multiplies that rate by the number of days, and returns the total price to the client.

The Vehicle **Season** Price microservice acts as the *data* layer. It loads pricing data from a JSON file at startup and exposes an endpoint that returns the daily rental rate for any valid vehicle type and season combination. This service is protected by HTTP Basic Authentication.

1.3 Configuration Management

Spring Cloud Config Server is used for centralized configuration management. Both microservices retrieve their configuration (port assignments and service URLs) from the Config Server at startup. The Config Server pulls configuration from a dedicated folder in the project's GitHub repository.

The configuration repository is located at:

```
https://github.com/bcibula/vehicle-rental-a2
```

Configuration files are stored in the `vehicle-config-repo/` subdirectory:

- `vehicle-season-price-a2.properties` — defines port 8100
- `vehicle-total-price-a2.properties` — defines port 8200 and the season-price service URL

2. Data

Pricing data is stored as a JSON file (vehicle_season.json) loaded by the Vehicle Season Price microservice at startup using Jackson's ObjectMapper. No database is required. The dataset contains five vehicle types with daily rental prices for four seasons.

Vehicle Type	Spring	Summer	Fall	Winter
Compact	\$45	\$50	\$43	\$40
Sedan	\$55	\$60	\$53	\$50
SUV	\$65	\$70	\$63	\$60
Convertible	\$75	\$80	\$73	\$70
Truck	\$85	\$90	\$83	\$80

Input values for vehicle type and season are normalized to title case before lookup, so requests using any combination of uppercase and lowercase characters are handled correctly (e.g., "suv" and "SUV" return identical results).

3. API Endpoint Documentation

3.1 Vehicle Season Price Service

Base URL: `http://localhost:8100`

Authentication: HTTP Basic Authentication required on all endpoints

Credentials: Username: `rental` | Password: `rental123`

GET /price

Returns the daily rental price for a specified vehicle type and season.

Parameter	Type	Required	Description
vehicle	String	Yes	Vehicle type (e.g., SUV, Compact, Truck)
season	String	Yes	Season (Spring, Summer, Fall, Winter)

Example Request:

```
GET http://localhost:8100/price?vehicle=SUV&season=Summer
```

Authorization: Basic Auth — username: rental, password: rental123

Example Response (200 OK):

```
{
  "vehicle": "SUV",
  "season": "Summer",
  "dailyRate": 70,
  "port": "8100"
}
```

Example Response — Invalid Vehicle (404 Not Found):

```
{
  "error": "No pricing found for vehicle: Bicycle in season: Summer",
  "port": "8100"
}
```

Example Response — Missing Credentials (401 Unauthorized):

```
(empty body — Spring Security returns 401 with WWW-Authenticate header)
```

3.2 Vehicle Total Price Service

Base URL: `http://localhost:8200`

Authentication: None required (public endpoint)

GET /total-price

Calls the Vehicle Season Price service to retrieve the daily rate, then calculates and returns the total rental cost for the specified number of days.

Parameter	Type	Required	Description
vehicle	String	Yes	Vehicle type (e.g., SUV, Compact, Truck)
season	String	Yes	Season (Spring, Summer, Fall, Winter)
days	Integer	Yes	Number of rental days

Example Request:

```
GET http://localhost:8200/total-price?vehicle=SUV&season=Winter&days=10
```

Example Response (200 OK):

```
{
  "vehicle": "SUV",
  "season": "Winter",
  "days": 10,
  "dailyRate": 60,
  "totalPrice": 600,
  "port": "8200"
}
```

4. Setup and Run Instructions

4.1 Prerequisites

- Java 17
- Maven 3.x
- IntelliJ IDEA (or any Java IDE)
- Internet connection (Config Server pulls from GitHub)

4.2 Clone the Repository

```
git clone https://github.com/bcibula/vehicle-rental-a2.git
```

4.3 Open Projects in IntelliJ

Open each of the three Spring Boot projects as separate IntelliJ projects by pointing the IDE at the subfolder containing the pom.xml:

- vehicle-rental-a2/config-server-a2
- vehicle-rental-a2/vehicle-season-price-a2
- vehicle-rental-a2/vehicle-total-price-a2

Allow Maven to finish importing dependencies in each project before proceeding.

4.4 Start the Services — Order Matters

The services must be started in this exact order:

Step	Service	Main Class	Port
1	Config Server	ConfigServerA2Application	8888
2	Vehicle Season Price	VehicleSeasonPriceA2Application	8100
3	Vehicle Total Price	VehicleTotalPriceA2Application	8200

The Config Server must be running before either microservice starts, as they connect to it on startup to retrieve their port configuration.

4.5 Verify Services Are Running

After starting each service, verify it is running by checking the following URLs in a browser:

Service	Verification URL	Expected Result
Config Server	http://localhost:8888/vehicle-season-price-a2/default	JSON with server.port: 8100
Config Server	http://localhost:8888/vehicle-total-price-a2/default	JSON with server.port: 8200
Season Price	http://localhost:8100/actuator/health	{"status":"UP"}
Total Price	http://localhost:8200/actuator/health	{"status":"UP"}

4.6 Testing the Endpoints

Use Postman or a browser to test the endpoints. Note that the Vehicle Season Price service requires Basic Authentication (username: rental, password: rental123). The Vehicle Total Price service requires no authentication.

Example test requests:

```
GET http://localhost:8100/price?vehicle=SUV&season=Summer (with Basic Auth)

GET http://localhost:8200/total-price?vehicle=SUV&season=Winter&days=10
```


5. Assumptions and Design Decisions

5.1 JSON Data Over Database

The assignment offered a choice between importing data into a database or using JSON directly. JSON was chosen because the data is static, no CRUD operations are required, and it avoids introducing an unnecessary database dependency. The JSON file is loaded into memory at service startup using Jackson's ObjectMapper.

5.2 Case Normalization

Vehicle type and season parameters are normalized to title case before lookup. This means a request with "suv" or "SUV" or "Suv" will all return the same result. Normalization is applied in the VehicleSeasonPriceService before the data lookup, keeping the controller and model clean.

5.3 Security Implementation

HTTP Basic Authentication is applied to the Vehicle Season Price microservice using Spring Security. Credentials are configured in application.properties (username: rental, password: rental123). The Vehicle Total Price microservice passes these credentials when making inter-service calls using Spring's RestTemplate with HttpHeaders.setBasicAuth().

The Vehicle Total Price microservice is not itself secured, as it is the entry point for end-user requests. In a production environment, an API gateway or OAuth2 token-based authentication would be used instead of Basic Auth.

5.4 Port Configuration via Config Server

All port assignments and inter-service URLs are externalized to the Spring Cloud Config Server rather than hardcoded in application.properties. This demonstrates configuration management as a core microservices principle — changing a port requires only a properties file update in the config repository, with no code changes required.

5.5 Spring Boot Version

This project uses Spring Boot 4.1.0 (Spring Framework 7.x, Jakarta EE 11), which is the current stable release at the time of submission. The course reference materials reference an older stack (Spring Boot 2.x, JAX-RS/CXF). All modern equivalents have been substituted where necessary.

6. GitHub Repository

All source code is available at:

```
https://github.com/bcibula/vehicle-rental-a2
```

Repository structure:

- `config-server-a2/` — Spring Cloud Config Server
- `vehicle-season-price-a2/` — Vehicle Season Price microservice
- `vehicle-total-price-a2/` — Vehicle Total Price microservice
- `vehicle-config-repo/` — Configuration properties files

7. AI Transparency and Acknowledgment

McGill AI Policy - 3. Transparency and Acknowledgment:

This assignment used AI assistance (Claude Sonnet 4.6, Anthropic) for: learning aid (understanding Spring Cloud Config Server behavior, inter-service communication patterns, RestTemplate usage, Spring Security Basic Authentication, and microservices architecture concepts); code debugging during development (resolving dependency mismatches between Spring Boot 4.x and course reference materials written for Spring Boot 2.x, and diagnosing inter-service HTTP call failures); and assembling the technical report document skeleton.